EEE104 Lab Report:

Digital Electronics

(version 1)

Felix Lianto (2362727)

November 15, 2024

1 Introduction

Digital circuits have become the backbones for a lot of electronics we use today. They are frequently used to translate voltage values into logic levels, which the designer could use to implement computer logic and build a computer from the ground up. Implementations of these circuits can be found within small, rudimentary gadgets up to general purpose computers and smartphones.

This laboratory report aims to:

1. Understand Boolean Algebra and its corresponding theorems.
2. Implement and observe basic logic gates individually and combinational logic circuits.
3. Implement and observe basic types of memory circuits.

1.1 Preliminary Information

The world works in analog systems, which means that their values can be measured and are always continuous [11]. This can be observed and analyzed using the methods of continuous mathematics, such as calculus, and experimental observations. However, these systems are not the easiest to work with, given their nature that things cannot be precisely exact when working with continuous values. This is where digital systems can be useful for those specific circumstances.

Digital systems only works under binary states, that is, only one of two possible values are allowed for any given digital system [11]. For example, a switch is a digital system as it can only be turned on or turned off, nothing else. In digital systems, only a range of values are important to analyze, not a precise value. This fundamental principle behind digital systems is the backbone behind all smart devices built today.

Logic Circuits: Basic Logic Gates, Boolean Algebra, and DeMorgan's Theorem

To convert random binary states into something meaningful, it is important to understand the significance of logic gates. Logic gates can operate on one or more input signals and output another binary signal based on its rules. As an example, consider the NOT gate. The NOT gate considers only one input signal and outputs the inverse of that signal. This means that if an input of OFF is fed, it outputs ON, and vice-versa. Indeed, these gates can be compounded into more complicated circuitry, hence the need for an algebraization to simplify these logical circuits.

This can be remedied by an algebraic structure known as Boolean Algebra. First introduced by George Boole in 1847 [1], this system can formally transform

a logical circuit into an algebraic expression, where the variables only have binary states. Table 1 shows the algebraic representation of the gates, while Figure 1 highlights some basic theorems from such a system.

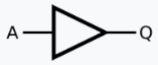
Logic Gate	Diagram of Logic Gate	Equivalent Boolean Expression
Buffer		A
NOT		$\overline{A}$
AND		AB
OR		$A + B$
NAND		$\overline{AB}$
NOR		$\overline{A + B}$
XOR		$\overline{A}B + A\overline{B}$
XNOR		$AB + \overline{A} \cdot \overline{B}$

Table 1: Fundamental logic gates, its circuit diagram (according to the IEEE Standard Graphic Symbols for Logic Functions or IEEE/ANSI 91/91a-1984), and its equivalent boolean expression [7].

From these theorems, we can prove important theorems that can help with simplification of boolean expressions. One such theorem is the DeMorgan's Theorem, commonly used to separate joint variables under a NOT operator [11];

Lemma 1 (Negation by Complementary Law [10])**.** *For any boolean variable* B*,* $A = \overline{B}$ *if and only if* $AB = 0$ *and* $A + B = 1$*.*

Proof. Suppose that $A = \overline{B}$. Clearly, by the complementary law, we have the desired result.

Theorem	Function	Dual
Idempotency	$A+A=A$	$A\cdot A=A$
Union & Intersection	$A+0=A$	$A\cdot 1=A$
	$A+1=1$	$A\cdot 0=0$
Complementation	$A+\overline{A}=1$	$A\cdot\overline{A}=0$
Inversion	$(\overline{\overline{A}})=A$	

Figure 1: Fundamental Theorems of Boolean Algebra [6]

Now suppose that $AB = 0$ and $A + B = 1$. Then, notice that

$$\begin{aligned} A + B &= 1 \\ (A + B)\overline{B} &= 1 \cdot \overline{B} \\ A\overline{B} + B\overline{B} &= \overline{B} \\ A\overline{B} &= \overline{B} \end{aligned}$$

Then, since we know that $AB = 0$, we have,

$$\begin{aligned} AB &= 0 \\ A\overline{B} + AB &= \overline{B} + 0 \\ A(B + \overline{B}) &= \overline{B} \\ A &= \overline{B} \end{aligned}$$

Thus, the lemma is proven [10]. ■

Lemma 2 (Absorption Law)**.** *For any boolean variable A and B, we have*

$$\begin{aligned} A + AB &= A \\ A + \overline{A}B &= A + B \end{aligned}$$

Proof. Notice that for the first statement,

$$\begin{aligned} A + AB &= A \cdot (1 + B) \\ &= A \end{aligned}$$

and for the second statement,

$$\begin{aligned} A + \overline{A} &= (A + \overline{A})(A + B) \\ &= 1 \cdot (A + B) \\ &= A + B \end{aligned}$$

Thus, the claim holds. ■

Theorem 1 (DeMorgan's Theorem)**.** *For any boolean variable A and B, we have*

$$\overline{AB} = \overline{A} + \overline{B} \tag{1}$$

$$\overline{A+B} = \overline{A} \cdot \overline{B} \tag{2}$$

Proof. (Statement 1) Notice, by Lemma 1, that it is sufficient to show that $AB + (\overline{A} + \overline{B}) = 1$ and $AB(\overline{A} + \overline{B}) = 0$ in order to prove statement 1. Observe that

$$\begin{aligned} AB + (\overline{A} + \overline{B}) &= \overline{B} + (AB + \overline{A}) && \text{(Rearrange)} \\ &= \overline{B} + (\overline{A} + B) && \text{(Lemma 2)} \\ &= \overline{A} + (B + \overline{B}) && \text{(Rearrange, Identity)} \\ &= \overline{A} + 1 = 1 && \text{(Complementation, Identity)} \end{aligned}$$

and also

$$\begin{aligned} AB(\overline{A} + \overline{B}) &= A\overline{A}B + AB\overline{B} && \text{(Distributivity)} \\ &= 0 \cdot B + A \cdot 0 && \text{(Inverses)} \\ &= 0 + 0 = 0 && \text{(Identity)} \end{aligned}$$

Therefore, the result follows. ■

Proof. (Statement 2) We can utilize the first statement to prove this second statement. Observe that

$$\begin{aligned} \overline{A+B} &= \overline{\overline{\overline{A} + \overline{B}}} && \text{(Inversion)} \\ &= \overline{\overline{\overline{A} \cdot \overline{B}}} && \text{(First statement)} \\ &= \overline{A} \cdot \overline{B} && \text{(Inversion)} \end{aligned}$$

Thus, the result is proven as required. ■

We will prove this theorem by truth tables in the Results section as well.

With the basic tools in mind, basic arithmetic can built based on this logic system through circuitry. One such logic circuits is known as the half-adder. The half-adder takes two inputs and performs addition on it, returning two output signals. The two output signals can be transcribed by utilizing binary numbers. A circuit diagram can be seen in Figure 2.

While this circuit can perform addition, it has its limitations. Namely, it can only do single bit calculations. It cannot support more than one binary numbers, such tasks are possible through using full adders [5]. Furthermore, it can also experience some delay in computing, rendering it not ideal for fast calculations should an input be altered [5].

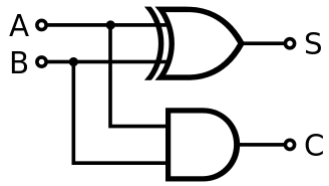


Figure 2: A half adder [9].

Memory Circuits: Latches, Flip-flops, Counters, and Shift Registers

One subset of logic circuits that will be explored in this report are memory circuits. Memory circuits are defined as logic circuits that can retain information from past inputs. The most common types of these circuits are latches and flip-flops.

Latches and flip-flops function identically the same, however there is one crucial difference between the two. Latches work by level-trigger, which indicates that it is only affected by the incoming input signals. Meanwhile, flip-flops are edge-triggered, which updates according to the input signals and also an external input signal commonly known as a "clock". Moreover, the edge-triggering mechanism can also be divided further into 2 categories, a rising edge trigger (updates when logic levels switch from low to high) and a falling edge trigger (updates when logic levels switch from high to low).

Flip-flops and latches can be divided into several types: SR (Set-Reset), D (Data), and JK (it's complicated). SR and JK functions the same, if a signal is passed through the S input, it will set the output to be HIGH, and if a signal is passed through R, it will set the output to LOW ("reset"). The only difference in those two types are that SR latches disallows for both S and R to be high, while JK does allow for both to be high, functioning as a "toggle" for the output. An illustration can be found within the truth table in Table 2.

S(et)	R(eset)	Q(output)	$\overline{Q}$(negated output)
0	0	No Change	No Change
1	0	1	0
0	1	0	1
1	1	INVALID	INVALID

J(set)	K(reset)	Q(output)	$\overline{Q}$(negated output)
0	0	No Change	No Change
1	0	1	0
0	1	0	1
1	1	Toggles	Toggles

Table 2: Truth table for SR (top) and JK (bottom) types

D types are an extension of the SR and JK types, as it toggles the output whenever it is inputted. Common latches and flip-flops circuit diagrams are shown in Table 3.

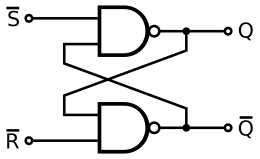
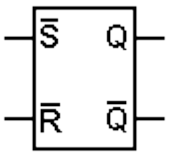
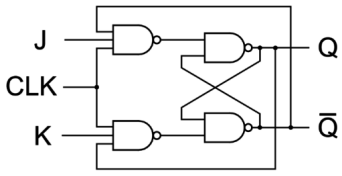
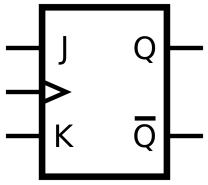
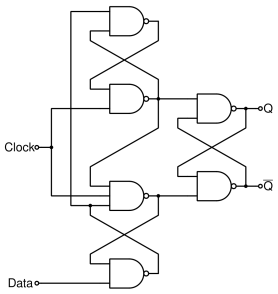
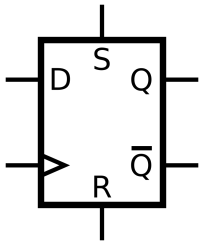
Memory Circuit	Implementation	Circuit Symbol
SR Latch		
JK Flip-flop		
D Flip-flop		

Table 3: Common memory circuit components, with their implementations and symbols [2, 4]

An application for flip-flops are counters, which are practically chained flip-flops, designed to count by increments of one in binary. Counters can be subdivided into 2 categories, asynchronous and synchronous. Asynchronous counters (or ripple counters [11]) are clocked in sequence of each flip-flops, meaning that for the next flip-flop to toggle, it needs to wait for the previous flip-flop. This is the one drawback of asynchronous counters, known as the propogation delay [11]. However, this can be solved with the second category of counters, synchronous counters. Synchronous counters (or parallel counters [11]) work using the same clock, as opposed to preceding flip-flop output. This leads to much faster counting which lets us handle more complicated tasks without being limited by delays [11]. Figure 3 and Figure 4 shows the implementation of these counters.

However, this does not imply that the asynchronous counter is useless. In

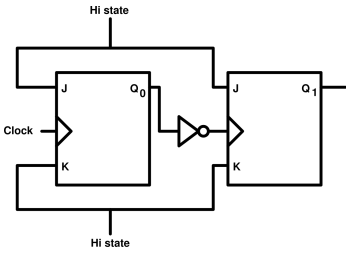


Figure 3: A 2-bit asynchronous counter.

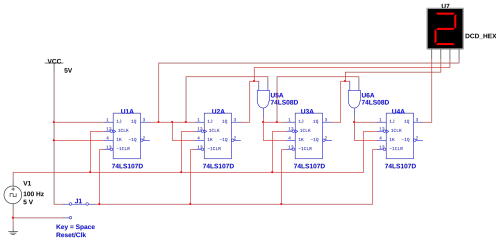


Figure 4: A 4-bit synchronous counter.

fact, the asynchronous counter could be used not as a counter, but as storage instead. This is known as the shift register. Shift registers are used for a lot of purposes, namely to convert data in series to parallel and hardware multiplication or division by 2 [12].

For this laboratory report, we will focus on a type of shift register called a ring counter. It is named so due to the last flip-flop output of the register is connected back to the first flip-flop, hence the "ring". This also implies that so long as there is a clock pulse, the cycle keeps repeating indefinitely [11].

Ring counters can also be subdivided into 2 types, the straight ring counter or the twisted ring counter [11]. Straight ring counters, also known as one-hot counter, are ring counters that passes its state onto the next flip-flop and resets itself, and is what specialists typically refer to when describing a ring counter. Twisted ring counters, also known as Johnson counters, are ring counters that doesn't start resetting itself until the last flip-flop is active. Circuit diagrams for both types are shown in Figure 5 and 6.

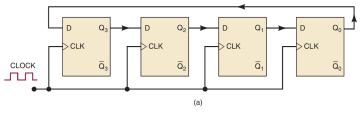


Figure 5: A 4-bit straight ring counter [11].

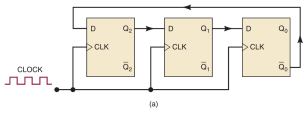


Figure 6: A 3-bit twisted ring counter [11].

2 Methodology

The laboratory session utilized the following apparatus to conduct the experiment;

1. NI Elvis III prototyping board and a computer running Windows 10, integrating power supplies, a function generator, and a multimetre.

2. Hook-up wires and 7400-series Low-power Schottky integrated circuits listed below. Definitions for the integrated circuits can be found in Appendix A.

   (a) 1 x 74LS00 (NAND Gate)

   (b) 1 x 74LS02 (NOR Gate)

   (c) 1 x 74LS04 (NOT Gate)

   (d) 1 x 74LS08 (AND Gate)

   (e) 1 x 74LS32 (OR Gate)

   (f) 1 x 74LS86 (XOR Gate)

   (g) 2 x 74LS74 (D Flip-flop)

   (h) 2 x 74LS112 (JK Flip-flop)

   (i) 4 x 330Ω Resistor

   (j) 4 x Light-emitting diode (LED) Lights

The experiment will be divided into 8 sections, and for each section, the experiment will utilize the NI Elvis III breadboard to connect the wires and logic gates. The gates will be supplied a $+5V$ voltage signaling HIGH and the ground voltage ($0V$) signaling LOW.

1. **Test of Basic Logic Gates**

   This section will examine voltage values between the output from 3 different types of logic gates (NOT gate, AND gate, and NOR gate) and the ground connection. Furthermore, the voltage values obtained will be generalized to a logic level and a truth table for each logic gate will be derived.

2. **DeMorgan's Theorem**

   This section will prove DeMorgan's theorem by connecting 2 circuits shown in Figure 7 and showing that their corresponding voltage values, and by extension, logic levels are equivalent. This will be proven through a truth table with all possible combinations of input.

3. **Combinational Logic Circuits**

   This section will implement a circuit based on the boolean expression $F = (A + B)(C + D)$ and a truth table will be derived based on the output signal.

4. **Implementation of a Half Adder**

   This section will implement the half-adder circuit shown in Figure 2, experimenting with its inputs, and report the output values in a truth table.

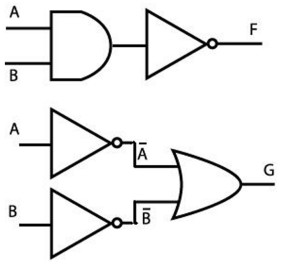


Figure 7: Two circuits to prove DeMorgan's Theorem [3].

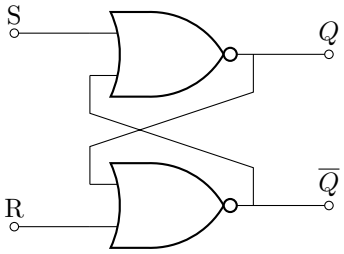


Figure 8: A NOR version of the SR latch [3].

5. **The S-R Latch**

   This section will implement the NOR version of the SR latch, shown in Figure 8. Afterwards, the circuit will be tested for all kinds of configurations, which will be documented in a truth table.

6. **The Edge-Triggered D Flip-flop**

   This section will implement the D Flip-flop shown in Figure 9. The truth table will be inferred and documented from this circuit.

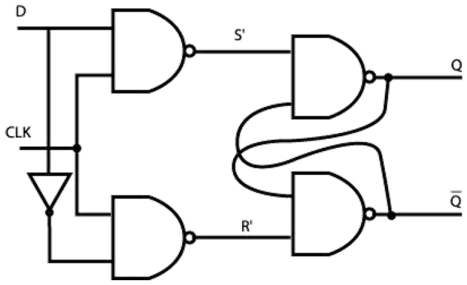


Figure 9: An edge-triggered D Flip-flop [3].

7. **The Asynchronous Counter**

This section will implement the asynchronous counter displayed in Figure 10. Then the flip-flops will all be reset and a clock pulse of 1 2 Hz on the function generated will be applied to the circuit. The corresponding output will be observed in a truth table.

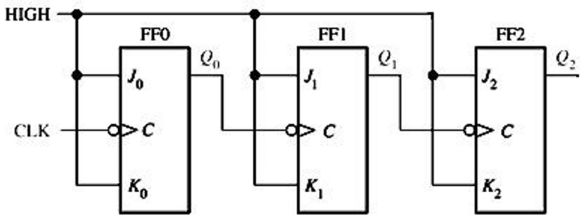


Figure 10: An asynchronous counter [3].

8. **The Shift Register**

This section will implement the Johnson shift register shown in Figure 11. A clock pulse will be registered to the circuit, similar to the previous section, and its resulting signal will be detailed in a truth table.

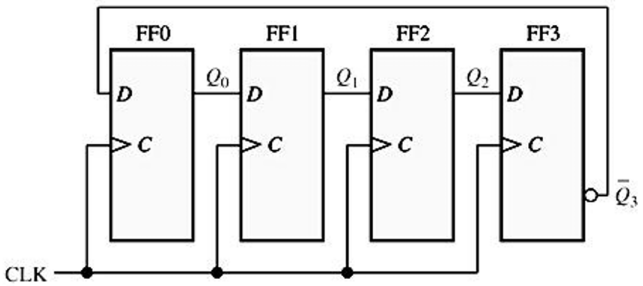


Figure 11: A Johnson shift register [3].

3 Results

3.1 Test of Basic Logic Gates

Table 4 shows the truth table for all 3 gates, both for expected values and experimental values.

Input		NOT Gate (Input A)		AND Gate		NOR Gate	
A	B	Expected	Actual	Expected	Actual	Expected	Actual
0 V	0 V	+5 V	+4.4257 V	0 V	+0.0005 V	+5 V	+4.4317 V
+5 V	0 V	0 V	+0.1603 V	0 V	+0.0005 V	0 V	+0.0003 V
0 V	+5 V			0 V	+0.0005 V	0 V	+0.0003 V
+5 V	+5 V			+5 V	+4.9965 V	0 V	+0.0001 V

Table 4: Truth Table for NOT, AND, and NOR gate.

3.2 DeMorgan's Theorem

Table 5 shows the truth table for both gates, as well as their expected values.

Inputs		NAND Gate		Negative OR Gate	
A	B	Expected	Actual	Expected	Actual
0 V	0 V	+5 V	+4.4324 V	+5 V	+4.9944 V
+5 V	0 V	+5 V	+4.4335 V	+5 V	+4.9938 V
0 V	+5 V	+5 V	+4.4335 V	+5 V	+4.9938 V
+5 V	+5 V	0 V	+0.0003 V	0 V	+0.0009 V

Table 5: Truth Table to prove DeMorgan's Theorem.

3.3 Combinational Logic Circuits

Table 6 shows the truth table for the given Boolean expression, $F = (A + B)(C + D)$.

Inputs				Output (F)	
A	B	C	D	Expected	Actual
0 V	0 V	0 V	0 V	0 V	+0.0001 V
+5 V	0 V	0 V	0 V	0 V	+0.0005 V
0 V	+5 V	0 V	0 V	0 V	+0.0005 V
+5 V	+5 V	0 V	0 V	0 V	+0.0003 V
0 V	0 V	+5 V	0 V	0 V	+0.0005 V
+5 V	0 V	+5 V	0 V	+5 V	+4.9986 V
0 V	+5 V	+5 V	0 V	+5 V	+4.9986 V
+5 V	+5 V	+5 V	0 V	+5 V	+4.9941 V
0 V	0 V	0 V	+5 V	0 V	+0.0005 V
+5 V	0 V	0 V	+5 V	+5 V	+4.9986 V
0 V	+5 V	0 V	+5 V	+5 V	+4.9986 V
+5 V	+5 V	0 V	+5 V	+5 V	+4.9941 V
0 V	0 V	+5 V	+5 V	0 V	+0.0003 V
+5 V	0 V	+5 V	+5 V	+5 V	+4.9941 V
0 V	+5 V	+5 V	+5 V	+5 V	+4.9941 V
+5 V	+5 V	+5 V	+5 V	+5V	+4.9937 V

Table 6: Truth table for the boolean expression in Section 3.

3.4 Implementation of a Half-adder

Table 7 shows the truth table of the implemented half-adder.

Inputs		Outputs			
[redacted]		Expected		Actual	
A	B	XOR Gate	AND Gate	XOR Gate	AND Gate
0 V	0 V	0 V	0 V	+0.1584 V	-0.0008 V
+5 V	0 V	+5 V	0 V	+4.4175 V	+0.0006 V
0 V	+5 V	+5 V	0 V	+4.4175 V	+0.0006 V
+5 V	+5 V	0 V	+5 V	+0.1681 V	+4.9874 V

Table 7: Truth table for a half-adder.

3.5 The SR Latch

Table 8 shows the truth table of an SR Latch, complete with previous states of the circuit.

Previous State		Inputs		Output			
[redacted]				Expected		Actual	
Q_n	$\overline{Q_n}$	S	R	Q_{n+1}	$\overline{Q_{n+1}}$	Q_{n+1}	$\overline{Q_{n+1}}$
0	1	0	0	0	1	0	1
1	0	0	0	1	0	1	0
0	1	0	1	0	1	0	1
1	0	0	1	0	1	0	1
0	1	1	0	1	0	1	0
1	0	1	0	1	0	1	0
0	1	1	1	Random	Random	0	0
1	0	1	1	Random	Random	0	0

Table 8: Truth table for the SR Latch.

3.6 The Edge-triggered D Flip-flop

Table 9 shows the truth table of an edge-triggered D flip-flop, along with the previous states of the circuit.

Previous State		Input	Output			
[redacted]			Expected		Actual	
Q_n	$\overline{Q_n}$	D	Q_{n+1}	$\overline{Q_{n+1}}$	Q_{n+1}	$\overline{Q_{n+1}}$
0	1	0	0	1	0	1
1	0	0	1	0	1	0
0	1	1	1	0	1	0
1	0	1	0	1	0	1

Table 9: Truth table for an edge-triggered D flip-flop.

3.7 The Asynchronous Counter

Table 10 shows the sequence table of the three output values of the flip-flops.

Note that the 8th and 9th tick are equal to the initial and 1st tick. This suggests that the counter is in an indefinite cycle.

[redacted]	Circuit State					
[redacted]	Expected			Actual		
Clock Cycle	Q_0	Q_1	Q_2	Q_0	Q_1	Q_2
Initial	0	0	0	0	0	0
1	1	0	0	1	0	0
2	0	1	0	0	1	0
3	1	1	0	1	1	0
4	0	0	1	0	0	1
5	1	0	1	1	0	1
6	0	1	1	0	1	1
7	1	1	1	1	1	1
8	0	0	0	0	0	0
9	1	0	0	1	0	0

Table 10: Sequence table for an asynchronous counter.

3.8 The Shift Register

Table 11 shows the sequence table of the Johnson Register at each clock step.

[redacted]	Circuit State							
[redacted]	Expected				Actual			
Clock Cycle	Q_0	Q_1	Q_2	Q_3	Q_0	Q_1	Q_2	Q_3
Initial	0	0	0	0	0	0	0	0
1	1	0	0	0	1	0	0	0
2	1	1	0	0	1	1	0	0
3	1	1	1	0	1	1	1	0
4	1	1	1	1	1	1	1	1
5	0	1	1	1	0	1	1	1
6	0	0	1	1	0	0	1	1
7	0	0	0	1	0	0	0	1
8	0	0	0	0	0	0	0	0

Table 11: Sequence table for a Johnson shift register.

Notice that this shares a similarity with the previous section, that is, it loops indefinitely.

4 Discussion

This section will be split into 2 sections, one for the general result gathered from the experiment and comparing it to the expected theoretical results, and another one for further discussions on selected sections, as required by the laboratory regulations.

4.1 General Result Discussion

The results from the preceding section matched what the expected output is. However, there are still discrepancy between the two values. Namely, the voltage output is not exact. This might be caused by the resistors residing in the integrated chip used for this laboratory. The expected voltages are ideal, meaning that an assumption was made to have "perfect resistors", meaning that the wires inside the logic chip had no resistance. However, the real world is far from ideal, therefore small errors like it exists.

Furthermore, there are more glaring value discrepancies found in the result of some sections of the experiment. This might be due to the fact that the measurement of the voltage was not perfect, or the implementation of the chip was crooked. This can cause a larger voltage difference from the actual voltage. However, this does not pose a problem for this experiment, as readings could still be inferred as HIGH or LOW for a suitable range.

The only incorrect hypothesis was found in the SR latch section. The original hypothesis was that if both S and R were HIGH, then the expected output would be random, hence why the need to ban such inputs. However, this hypothesis was not correct. What the circuit actually does is that when both are set to HIGH, both resulting outputs are set to LOW [8]. The reason behind why this is not allowed is when both S and R return back to LOW [8]. In which case, propogation delay will decide which switch would turn on, which for the human observer, would essentially be randomized [8].

4.2 Particular Sections Discussion

In section 2, we have proven the DeMorgan's Theorem by physically implementing the equivalent circuits and examined the output for both circuits. To prove that its boolean expression are equivalent, it can be showed that the first circuit has the boolean expression $\overline{AB}$, while the second circuit has the expression $\overline{A} + \overline{B}$. Since it has been proven that the first and second circuit are equivalent by the truth table in Table 5, we can conclude that $\overline{AB} = \overline{A} + \overline{B}$, as required.

In section 4, our experimental results matched our expected results, meaning arithmetic is possible with this circuit. However, as discussion in the Preliminary Information section, half-adders are not sufficient to implement basic binary arithmetic, as it can only handle single bit addition, not sufficient for carry calculations. Most complex digital circuits almost always needs a binary

calculator that can handle more than one bit and the possibility of chaining carry bits (i.e. 0111 to 1000).

In section 5, we implemented the SR latch and observed its inputs based on previous states and the S and R input. However, to explain in further detail on why the circuit behaves like expected, it is important to understand the step-by-step process behind it.

Regardless of if the circuit has been initialized or not, the output for an SR latch is forcibly decided on if either S or R is set to HIGH. In the case that S and R are both set to HIGH, then both outputs will return LOW. This is what was observed in the results section above, and more details regarding both S and R being HIGH were explained in the previous subsection.

However, the more interesting behavior of this circuit is when both S and R are low. If the circuit is initialized in this configuration, at least one of the output will be HIGH. This decision is made purely at random based on speed and propogation delay, as discussed previously. Then, once either one is set to HIGH, if both inputs are LOW then it will stay at that state, which is what the intersection wires in the middle are for. The circuit "remembers" the state, thus, it is a memory circuit.

In section 6, the experiment investigated the D flip-flop and its output based on the D input and the previous states. Much like the preceding section related to the SR latch, we will also try and explain how the D flip-flop works.

The D flip-flop is essentially an extension of the SR latch. However, it is complicated by the 2 NAND gates and the clock signal before it. The D signal is responsible for deciding if the output should be set to HIGH or LOW. If D is LOW, then output is set to LOW, and vice-versa. This is what is found in the truth table above. It can store this data because of the clock, as the clock is being actively blocked by the D signal in the first two NAND gates. D flip-flops have applications in shift registers, which we will discuss later in this report.

In section 7, the experiment implemented a 3-bit asynchronous counter and documented its results in the truth table in the results section. Generalizing the results, we can see that for an n-bit counter, we can count up to 2^n. The range for this count goes from 0 to $2^n - 1$.

This circuit can count because of the cascading flip-flops utilized here. To see this generally, suppose that k is an integer between 2 and n. We can see that the k-th flip-flop cannot toggle if the $k - 1$-th flip-flop doesn't provide an update in the form of the clock. Since k can be any value between that range, all the flip-flops can only be updated if the first flip-flop updates periodically. Thus, counting can be achieved.

There is another type of counters called the synchronous counters, which we have explained in the preliminary information section of this report. To summarize, synchronous counters don't rely on the first flip-flop to update. Instead, the same clock signal is connected to every circuit, with the output for one flip-flop connected to the following flip-flops.

5 Conclusion

This report has demonstrated and verified some concepts of digital circuits that are instrumental to nearly every digital devices present. Namely, we have discussed logic gates and Boolean Algebra, as well as combinational logic circuits and memory circuits.

While a successful laboratory experience was achieved, there are still some shortcomings that needed to be addressed for future sessions. First is regarding group work in the context of the laboratory. While a clear division of tasks was implemented, understanding of the material may not be equal amongst the laboratory participants. Thus, it is suggested that for future sessions, group members for the session are encouraged to try their hands on doing the field work, as opposed to documenting the results.

Moreover, this report was the author's third experience writing in LaTeX, and more features and extensions of this software could be used to professionalize this report by using specialized tools to draw figures and diagrams. Software such as CircuiTikz could have been used to produce higher quality circuit figures, however a lack of experience has made it difficult and time-consuming for the author.

This laboratory session has been enlightening and has reinforced the theoretical knowledge presented in the lectures and assignments. More opportunities for students to try and design logic circuits should stimulate their knowledge and desire to learn more about this subject.

Bibliography

[1] George Boole. *Logic: The Mathematical Analysis of Logic, Being an Essay Towards a Calculus of Deductive Reasoning*. CreateSpace Independent Publishing Platform, Nov. 3, 2015. 90 pp. ISBN: 978-1-5191-1866-0. Google Books: q6wbjwEACAAJ.

[2] Ajay Dheeraj. *J K Flip Flop Explained in Detail*. DCAClab Blog. Jan. 20, 2020. URL: https://dcaclab.com/blog/j-k-flip-flop-explained-in-detail/ (visited on 11/14/2024).

[3] *EEE104 - Digital Electronics Laboratory Manual*. Suzhou, China: Xi'an Jiaotong-Liverpool University, 2024. 11 pp.

[4] *Flip-Flop (Electronics)*. In: *Wikipedia*. Oct. 4, 2024. URL: https://en.wikipedia.org/w/index.php?title=Flip-flop_(electronics)&oldid=1249382557#JK_flip-flop (visited on 11/14/2024).

[5] *Half Adder in Digital Logic*. GeeksforGeeks. Aug. 3, 2015. URL: https://www.geeksforgeeks.org/half-adder-in-digital-logic/ (visited on 11/14/2024).

[6] B. Holdsworth and R.C. Woods. "Boolean Algebra". In: *Digital Logic Design*. Elsevier, 2002, pp. 28–42. ISBN: 978-0-7506-4582-9. DOI: 10.1016/B978-075064582-9/50003-2. URL: https://linkinghub.elsevier.com/retrieve/pii/B9780750645829500032 (visited on 11/14/2024).

[7] *Logic Gate*. In: *Wikipedia*. Oct. 28, 2024. URL: https://en.wikipedia.org/w/index.php?title=Logic_gate&oldid=1253919527 (visited on 11/14/2024).

[8] Neil_UK. *Answer to "What Actually Happens When Both 1 Input Is given in RS Flip Flop Circuit (Physical Change)?"*. Electrical Engineering Stack Exchange. Aug. 14, 2017. URL: https://electronics.stackexchange.com/a/323863 (visited on 11/15/2024).

[9] Margaret Rouse. *Half Adder*. Techopedia. Nov. 13, 2020. URL: https://www.techopedia.com/definition/7509/half-adder (visited on 11/14/2024).

[10] shinobi. *Answer to "Verify Demorgan's Law Algebraically"*. Mathematics Stack Exchange. Apr. 30, 2017. URL: https://math.stackexchange.com/a/2259353 (visited on 11/14/2024).

[11] Neal S. Widmer, Gregory L. Moss, and Ronald J. Tocci. *Digital Systems: Principles and Applications*. Harlow: Pearson, 2018. 1024 pp. ISBN: 978-1-292-16200-3.

[12] Ming Xu. "EEE104 Digital Circuits Lecture 18" (Xi'an Jiaotong Liverpool University).